

Aquitetura de Computadores — DGEMM

Matheus da Silva Conceição — DRE 125050534

¹PESC — Universidade Federal do Rio de Janeiro (UFRJ)

matheuswnscf@poli.ufrj.br

1. Introdução

O objetivo deste trabalho é implementar algoritmos de multiplicação de matrizes para demonstrar as ideias do livro *Computer Organization and Design: RISC-V* de Hennessy e Patterson. A cada capítulo do livro, um algoritmo de multiplicação de matrizes é implementado e cada um desses algoritmos possui uma otimização específica.

2. Algoritmo 1: Multiplicação de matrizes tradicional

2.1. Python puro

O algoritmo tradicional de multiplicação de matrizes é o mais simples e direto, onde cada elemento da matriz resultante é calculado como a soma dos produtos dos elementos correspondentes das linhas da primeira matriz e das colunas da segunda matriz. A implementação em Python puro pode ser feita utilizando três loops aninhados para iterar sobre as linhas e colunas das matrizes.

```
for i in xrange(n):
    for j in xrange(n):
        for k in xrange(n):
            C[i][j] += A[i][k] * B[k][j]
```

Figura 1. Implementação do algoritmo em Python.

Como é um dos mais simples, o algoritmo tradicional tem uma complexidade de tempo de $O(n^3)$, o que o torna ineficiente para matrizes grandes. Além disso, para esse algoritmo, em mais baixos níveis, para executar cada multiplicação, o processador precisa acessar a memória para obter os valores das matrizes inúmeras vezes. Outra questão também é o que os dados do Python puro (como o Float) contam com objetos, isso somado a fato das listas serem estruturas de dados mais complexas (listas dentro de listas), torna o processo de multiplicação ainda mais lento.

2.2. Numpy (biblioteca do Python)

Já o algoritmo de multiplicação de matrizes utilizando a biblioteca Numpy (uma biblioteca com foco em computação numérica e álgebra linear) é muito mais rápido e eficiente. A biblioteca Numpy é otimizada para operações de matrizes e utiliza implementações em C (como listas contíguas), o que deixa a multiplicação de matrizes muito mais rápida do que a implementação em Python puro.

3. Algoritmo 2: Multiplicação de matrizes utilizando C

A implementação do algoritmo de multiplicação de matrizes utilizando a linguagem C é significativamente mais rápida do que a implementação em Python puro. Primeiramente, diferentemente do Python puro, **em C os dados das matrizes são armazenados em arrays contíguos na memória**, ou seja, os dados na memória estão organizados de forma sequencial, o que permite um acesso mais eficiente à memória. Além disso, a linguagem C é compilada, o que significa que o código é traduzido diretamente para código de máquina, eliminando a etapa de interpretação presente no Python. Isso resulta em uma execução mais rápida, especialmente para operações intensivas em cálculos, como a multiplicação de matrizes.

```
1. void dgemm (int n, double* A, double* B, double* C)
2. {
3.     for (int i = 0; i < n; ++i)
4.         for (int j = 0; j < n; ++j)
5.             {
6.                 double cij = C[i+j*n]; /* cij = C[i][j] */
7.                 for( int k = 0; k < n; k++ )
8.                     cij += A[i+k*n] * B[k+j*n]; /* cij += A[i][k]*B[k][j] */
9.                 C[i+j*n] = cij; /* C[i][j] = cij */
10.            }
11. }
```

Figura 2. Implementação do algoritmo em C.

4. Algoritmo 3: otimização AVX

A otimização AVX (Advanced Vector Extensions) é uma extensão do conjunto de instruções x86 que permite a execução de operações vetoriais em paralelo, isso através de um conceito chamado SIMD (Single Instruction, Multiple Data) ou “subword parallelism”, definição que o livro texto utiliza.

Em vez de processar um elemento de cada vez, a otimização AVX permite que o processador execute a mesma operação em vários elementos simultaneamente, utilizando registradores vetoriais de 256 ou 512 bits.

```

1. //include <x86intrin.h>
2. void dgemm (size_t n, double* A, double* B, double* C)
3. {
4.     for ( size_t i = 0; i < n; i+=4 )
5.         for ( size_t j = 0; j < n; j++ ) {
6.             __m256d c0 = _mm256_load_pd(C+i+j*n); /* c0 = C[i][j] */
7.             for( size_t k = 0; k < n; k++ )
8.                 c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.                                     _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.                                                    _mm256_broadcast_sd(B+k+j*n)));
11.             _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.         }
13. }

```

Figura 3. Implementação da otimização AVX.

Primeiramente, a declaração `_mm256d` é um tipo de dado específico para armazenar vetores de 256 bits, que podem conter quatro números de ponto flutuante de precisão dupla (double). A função `_mm256_loadu_pd` é usada para carregar os dados consecutivos (por exemplo, 4 elementos de uma única linha ou coluna) das matrizes A e B para os registradores vetoriais, permitindo que as operações sejam realizadas em paralelo. A função `_mm256_mul_pd` multiplica os elementos correspondentes dos vetores, e a função `_mm256_add_pd` é usada para acumular os resultados da multiplicação. Por fim, a função `_mm256_storeu_pd` armazena o resultado final da multiplicação de matrizes de volta na matriz C.

O bom desempenho da otimização AVX se deve à capacidade de processar múltiplos elementos em paralelo, reduzindo o número de ciclos de clock necessários para realizar a multiplicação de matrizes.

5. Algoritmo 4: Otimização AVX com Loop Unrolling

Avançando para uma implementação mais sofisticada, a otimização passa a combinar o Paralelismo no Nível de Dados (SIMD) com o Paralelismo no Nível de Instrução (ILP). O diferencial desta etapa é a aplicação da técnica de *Loop Unrolling* (desenrolamento de laço) com um fator de 4. Em vez de utilizar apenas um vetor acumulador, é declarado um arranjo de registradores `_mm256d c[UNROLL]`. Isso expõe para o processador quatro operações matemáticas completamente independentes a cada iteração do laço.

```

1. #include <x86intrin.h>
2. #define UNROLL (4)
3.
4. void dgemm (int n, double* A, double* B, double* C)
5. {
6.     for (int i = 0; i < n; i+=UNROLL*8)
7.         for (int j = 0; j < n; ++j){
8.             m512d c[UNROLL];
9.             for (int r=0;r<UNROLL;r++)
10.                c[r] = _mm512_load_pd(C+i+r*8+j*n); //[ UNROLL];
11.
12.             for( int k = 0; k < n; k++ )
13.             {
14.                 m512d bb = _mm512_broadcastsd_pd(_mm_load_sd(B+j*n+k));
15.                 for (int r=0;r<UNROLL;r++)
16.                     c[r] = _mm512_fmadd_pd(_mm512_load_pd(A+n*k+r*8+i), bb, c[r]);
17.             }
18.
19.             for (int r=0;r<UNROLL;r++)
20.                 _mm512_store_pd(C+i+r*8+j*n, c[r]);
21.         }
22.     }

```

Figura 4. Implementação da otimização AVX com Loop Unrolling.

Para otimizar ainda mais o gargalo de acesso à memória, a instrução `_mm256_broadcast_sd` é empregada para ler um único elemento escalar da matriz B e replicá-lo de forma imediata por todo um registrador vetorial. O compilador reaproveita esse mesmo registrador nas multiplicações subsequentes. A operação matemática central é feita através da instrução `_mm256_fmadd_pd` (Fused Multiply-Add), que realiza a multiplicação dos vetores de A e B e o acúmulo em C em uma única instrução de hardware de alta velocidade.

O bom desempenho dessa abordagem ocorre porque o código fornece à arquitetura superescalar da CPU um fluxo contínuo de instruções. Graças à independência de dados nos registradores acumuladores e à capacidade de execução fora de ordem do processador, os cálculos ocorrem em paralelo contínuo, evitando bolhas de ociosidade (*stalls*) no *pipeline*.

6. Algoritmo 5: Otimização com Cache Blocking

O próximo passo na otimização da função DGEMM foca em transpor o gargalo da hierarquia de memória do processador, introduzindo a técnica de *Cache Blocking*. Quando matrizes de grandes dimensões são multiplicadas utilizando a abordagem tradicional, o volume de dados excede rapidamente a capacidade das memórias cache de nível 1 (L1) e 2 (L2). Isso provoca uma alta taxa de *cache misses*, forçando a CPU a buscar dados repetidamente na memória principal (RAM), penalizando o desempenho de forma severa.

Para mitigar esse problema, a implementação foi refatorada para iterar sobre a matriz em submatrizes quadradas menores, cujo tamanho é definido pela constante `BLOCKSIZE = 32`. Através de laços de repetição externos adicionais, o algoritmo particiona o problema, enquanto a função auxiliar `do_block` concentra-se em resolver a multiplicação exclusivamente para aquele pequeno bloco. Como este subconjunto de dados de 32x32 elementos cabe inteiramente na cache L1, as operações matemáticas reutilizam os valores já carregados na memória de acesso ultrarrápido (maximizando o reuso temporal e espacial da cache).

7. Especificações e métricas de teste

7.1. Especificações do ambiente de teste

- Processador: Ryzen 7 5700X (8 núcleos, 16 threads, 3.8 GHz base, 4.6 GHz boost)
- Placa de vídeo: AMD Radeon RX 6750 XT
- Memória RAM: 32 GB DDR4
- Sistema Operacional: Linux Ubuntu
- Compilador: GCC 13.3.0
- Python: 3.12.3
- SSD: 1 TB NVMe
- Todos os testes foram realizados utilizando o mesmo ambiente para garantir a consistência dos resultados: PC em Idle e sem outros processos em execução.

7.2. Métricas de teste

- Tempo de execução: O tempo total gasto para multiplicar matrizes de diferentes tamanhos.
- Tamanho das matrizes: Matrizes quadradas de tamanhos 512×512 , 1024×1024 , 2048×2048 , 4096×4096 e 8192×8192 .
- Testes repetidos: Cada teste foi executado 5 vezes para cada tamanho de matriz, e a média dos tempos de execução foi calculada para garantir a precisão dos resultados.
- Desvio padrão: O desvio padrão dos tempos de execução foi calculado para avaliar a variabilidade dos resultados.
- Máximo e mínimo: Os tempos máximo e mínimo de execução foram registrados para cada teste para identificar possíveis outliers.
- Comparação de desempenho: Os tempos de execução dos diferentes algoritmos foram comparados para avaliar o impacto das otimizações implementadas.

8. Análise de resultados

Os gráficos mostram a evolução das otimizações. Fica evidente que o Python puro é inadequado para computação intensiva, justificando a necessidade de linguagens de baixo nível. A versão em C sem otimizações (apenas compilada) já traz um ganho, mas é a combinação de SIMD (AVX), ILP (Loop Unrolling) e Localidade de Memória (Cache Blocking) que melhora de fato a performance.

Pontos de destaque

8.1. Gargalo de Memória

O algoritmo tradicional (AVX + Loop Unrolling) sofre com o gargalo de memória, pois cada multiplicação exige múltiplos acessos à RAM, resultando em um grande número de *cache misses*. A otimização com Cache Blocking reduz significativamente esse problema, mantendo os dados necessários na cache L1 e melhorando o desempenho. Isso fica evidente nos gráficos, onde para uma matriz pequena o algoritmo alcança um desempenho de 9.8 GFLOPS, mas para matrizes maiores o desempenho cai drasticamente.

8.2. Numpy

A biblioteca Numpy, apesar de ser escrita em Python, teve um desempenho bem satisfatório. Isso se deve ao fato de que a biblioteca é otimizada para operações de matrizes e utiliza implementações em C, o que permite um acesso mais eficiente à memória e uma execução mais rápida do que a implementação em Python puro, essas otimizações deram um ganho que permitiram ultrapassar a performance do algoritmo tradicional em C sem otimizações e até com otimização AVX.